SecDevQA: Can LLM Agents Answer Security Questions Developers Ask?

Anonymous Authors

***Abstract*—Developers routinely ask security questions in open-source project forums: whether a reported vulnerability affects their specific usage, which version introduced a regression, or whether a proposed patch is sufficient. Answering such questions often requires consulting multiple artifact types — source code, commit history, CVE and GHSA advisories, dependency manifests, or external documentation — yet it is unclear which artifact types are necessary for which categories of question, or how well current LLMs and agents perform under realistic context constraints. We present SecDevQA, a benchmark of [N] developer security question-answer pairs mined from security-labeled issues, advisory discussions, and CVE-linked threads across [K] open-source GitHub repositories. We restrict the benchmark to pairs where the maintainer's answer contains at least one externally verifiable factual claim — a CVE or GHSA identifier, fixed version number, fix commit SHA, or advisory record. These hard facts anchor automated grading: an LLM judge is prompted to verify whether a model's response correctly states each specific factual claim, a narrowly scoped task whose reliability can be directly validated by human spot-check against the structured ground truth. Each pair is annotated with the artifact types the maintainer's answer drew on (source code, commit history, advisory records, dependency manifests, documentation) and assigned to an inductively derived question category. We evaluate frontier and open-weight LLMs and coding agents on SecDevQA under three controlled context conditions — no context, single-artifact, and multi-artifact — and measure accuracy per question category and per artifact type. Our main empirical contribution is a map of which artifact types are necessary to answer which categories of developer security questions, and the marginal accuracy gain each type provides.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Developers regularly confront security decisions in the course of building and maintaining their own software. They need to know whether a freshly disclosed vulnerability in a dependency actually affects the way they use it, which release first introduced a regression, whether a proposed patch is sufficient, or which version is safe to upgrade to. When the answer is not obvious, they ask—on issue trackers, in advisory discussions, and on CVE-linked threads of the projects they depend on. How developers resolve such questions has direct consequences for the security of the software they ship: a substantial body of usable-security research shows that developers lean heavily on online information sources when solving security problems, and that the quality of those sources measurably shapes the security of the resulting code. Acar et al. found that only a small fraction of the Stack Overflow threads developers consult for security tasks contain secure advice [1], and Fischer et al. traced insecure snippets from such threads into hundreds of thousands of shipped applications [2]. Getting trustworthy answers to developer security questions is therefore not a convenience problem but a software-security problem.

These questions are also distinctive in what it takes to answer them. Unlike general code-comprehension questions, a developer security question typically cannot be resolved from the repository's source alone. Answering "does CVE-2022-25883 affect me, and what do I upgrade to?" requires cross-referencing artifacts that live partly outside the codebase: the CVE or GHSA advisory and its affected-version ranges, the fix commit or pull request, the project's dependency manifest, and sometimes an upstream RFC or registry. Project maintainers answer these questions precisely because they know which artifact to consult—and, crucially, *different questions demand different artifacts*. A question about a transitive dependency is answered from the advisory and the manifest; a question about a TLS validation change is answered from the source and the commit history. Which artifact types are necessary for which kinds of security question is, to date, an unmeasured empirical question.

Large language models (LLMs) and coding agents are increasingly the first responders to such questions, yet no existing benchmark evaluates them on the security questions developers actually ask, or on the artifact dependence those questions exhibit. Repository-level QA benchmarks—SWE-QA [3], CoReQA [4], and StackRepoQA [5]—treat developer questions uniformly and consider only code-internal context; they neither isolate security questions nor model the external advisory artifacts those questions depend on. (StackRepoQA further reports that much of the apparent accuracy on such tasks reflects memorization of public threads rather than reasoning [5], a contamination risk any security-QA benchmark must confront.) Security-specific LLM benchmarks, in turn, measure a different task: SecVulEval [6] and CodeSecEval [7] evaluate vulnerability detection and secure code generation, SecureAgentBench [8] evaluates whether agents write secure patches, and SecQA [9] tests textbook security knowledge via multiple-choice questions. None of them studies real, free-text developer security questions, and none treats artifact context as a controlled variable.

We close this gap with **SecDevQA**, a benchmark of real developer security question–answer pairs mined from GitHub issues, discussions, and CVE-linked threads across open-source GitHub repositories. Two design choices make the benchmark both rigorous and analytically useful. First, we restrict the benchmark to *hard-verifiable* pairs: pairs whose maintainer answer states at least one externally checkable

fact—a CVE or GHSA identifier, a fixed version number, a fix commit or pull request, or an advisory URL. These hard facts anchor automated grading, turning the otherwise subjective task of scoring a free-text answer into the narrowly scoped task of checking whether a model's response correctly conveys a named, verifiable claim—a judgment a human can validate in seconds. Second, every pair is annotated with the artifact types its answer drew on, so that artifact context can be supplied to models as a controlled experimental condition rather than left implicit.

This design lets us pursue three questions that prior work cannot. We first derive, by inductive open coding of the mined corpus, a taxonomy of developer security question categories and an empirical map of which artifact types maintainers draw on to answer each—findings about developer security practice that stand independently of any model evaluation (RQ1). We then measure how accurately frontier LLMs, open-weight LLMs, and coding agents answer these questions, broken down by category (RQ2). Finally, and centrally, we ask which artifact types are *necessary* to answer which categories of question, by supplying artifacts as context under controlled conditions and measuring each type's marginal contribution (RQ3); we further examine whether autonomous agents retrieve the artifacts they actually need (RQ4). Our guiding hypothesis is that context dependence is category-specific: an artifact type that is essential for one category of security question yields little benefit for another—a result with direct implications for how retrieval-augmented and agentic security tools should be built.

Contributions. This paper makes the following contributions:

- **SecDevQA**, a benchmark of real, hard-verifiable developer security Q&A pairs mined from open-source repositories, each annotated with the artifact types needed to answer it and a structured record of its verifiable hard facts (§II).
- A fact-grounded grading protocol that exploits hard facts to make LLM-judge scoring narrowly scoped and human-validatable (§II).
- An inductively derived taxonomy of developer security question categories and an empirical category-to-artifact map, both independent of any LLM evaluation (§II-E).
- An evaluation of frontier and open-weight LLMs and coding agents under controlled context conditions, quantifying the marginal, category-specific contribution of each artifact type (§III).
- A public release of the benchmark, the mining and extraction pipeline, and the evaluation harness.

II. Benchmark Construction

SecDevQA is built in four stages: (i) systematic selection of source repositories from the GitHub Advisory Database, (ii) LLM-assisted detection and extraction of candidate security question–answer (Q&A) pairs from issue and discussion threads, (iii) manual verification of each candidate pair, and (iv) inductive open coding of the verified pairs toward a taxonomy of developer security questions. The numbers reported below describe the corpus at the current stage of construction; the benchmark is being actively expanded along the same protocol, so absolute counts will grow while the procedure remains fixed.

A. Repository Selection

To avoid hand-picking projects in a way that could bias the question distribution, we draw the source repositories from an external, independently maintained catalogue of security activity: the GitHub Advisory Database. We clone the database (snapshot **2026-06-04**; **31,213** reviewed advisories) and parse every reviewed GHSA entry, extracting the GitHub repositories referenced by each advisory and, for each repository, the number of advisories that carry a documented fix version. This yields **9,102** distinct repositories that have appeared in at least one reviewed advisory.

We then apply three pre-specified filters intended to keep repositories that are both security-active and practically minable:

1) **Security activity:** at least **10** advisories with a documented fix version, so that the project has a substantive, externally catalogued security history;
2) **Popularity:** at least **1,000** GitHub stars, as a proxy for a real user base whose developers ask questions;
3) **Maintenance:** a push within the last **24** months and not archived, so that threads reflect current practice.

Applying the advisory-count filter leaves **440** repositories; adding the popularity and maintenance filters leaves **344**. To enforce language and ecosystem diversity rather than concentrating on whichever ecosystem happens to have the most advisories, we rank the survivors by fix-bearing advisory count within each package ecosystem (npm, PyPI, RubyGems, etc.) and retain the top repositories per ecosystem, producing a diverse candidate pool spanning **ten** ecosystems.

From this pool we are mining repositories incrementally. The current corpus comprises the **ten** repositories in Table I, spanning **three** languages (Python, JavaScript, Ruby) and a range of security-sensitive domains: HTTP clients (`requests`, `urllib3`, `axios`), cryptography (`pyca/cryptography`), image processing (`Pillow`), web frameworks (`express`, `fastapi`, `rails`), authentication (`node-jsonwebtoken`), and payment integration (`stripe-node`). The selection deliberately mixes repositories with a large catalogued advisory history with widely used libraries in domains where security questions are common but formal advisories are sparser, so that the corpus is not skewed toward projects with heavyweight advisory processes (§II-C describes the per-repository cap that enforces this).

B. Detection and Extraction of Q&A Pairs

For each repository we retrieve the most recent **2,000** issue and discussion threads and flatten them into ordered comment sequences. We bound the window deliberately: our goal is not to harvest every security Q&A pair a project has ever produced, but to collect a corpus large and diverse enough to support the taxonomy and evaluation in this study. Recent

threads also better reflect current libraries, advisories, and developer practice.

Locating security Q&A pairs in this material by hand is impractical. A single active repository accumulates thousands of issues, the large majority of which are ordinary functional bugs, feature requests, or support questions with no security content; security discussions are a sparse minority scattered across that volume, and confirming whether a given thread contains a genuine question–answer exchange requires reading it end to end. Manually browsing every issue across **ten** repositories—tens of thousands of threads—to find this minority is prohibitively slow. We therefore use an LLM as a *prefilter* that reads every thread and surfaces likely candidates, turning an intractable full-text triage into a tractable verification problem over a small candidate set. A two-stage LLM pipeline identifies candidate Q&A pairs.

Stage 1 (detection). An LLM reads each thread and decides whether it contains a genuine developer security question paired with a substantive answer, emitting a boolean, a short summary, and a confidence score. Because this stage is a prefilter feeding human verification, we deliberately prompt it to favor *recall* over precision: the cost of a false positive is a few seconds of reviewer time at the verification stage, whereas a false negative silently drops a real pair from the corpus with no opportunity for recovery. We accordingly set a permissive **0.70** confidence threshold and instruct the model to surface borderline threads rather than discard them. The downstream **39%** rejection rate (§II-C) is the expected, acceptable price of this recall-oriented design.

Stage 2 (extraction). For threads passing Stage 1, a second prompt extracts the specific question and answer comments, the role of the answerer (maintainer, contributor, commenter, or self-answer), the artifact types the answer draws on (§II-D), and any externally verifiable *hard facts* the answer states—CVE/GHSA/OSV identifiers, fixed version numbers, fix commit SHAs, fix PR references, and advisory URLs.

This pipeline produced **260** candidate pairs across the **ten** repositories. We intentionally extract candidates broadly at this stage and rely on manual verification, rather than the LLM, to make the final inclusion decision.

C. Manual Verification

Every candidate pair is reviewed by the first author through a purpose-built annotation interface that presents the full thread alongside the extracted question, answer, role, artifacts, and hard facts. A pair is *accepted* only if it satisfies the inclusion criteria: the question concerns a security situation in the developer's own project, the answer is substantive and information-providing, and the question and answer are attributable to distinct, identifiable comments. Pairs that are not genuinely security-related (e.g. ordinary functional bugs), that are unanswerable without information absent from the thread, or that are mis-extracted are *rejected*; the reviewer records a short reason for each rejection.

To prevent the most security-active projects from dominating the corpus, we cap the number of verified pairs retained per repository at **25**. This keeps the corpus balanced across projects, languages, and security domains rather than skewed toward the few repositories with the densest advisory history. In the current corpus the cap does not yet bind—every repository falls at or below it (the largest, `requests`, contributes **24** pairs)—but it fixes the composition policy as mining scales to more and larger repositories.

Of the **260** candidates, **158** (**60.8%**) were accepted and **102** were rejected—a rejection rate that, given the recall-oriented prefilter, is expected and underscores why LLM extraction alone is insufficient and human verification is required. No single repository exceeds **15.2%** of the verified pairs. The accepted pairs are answered predominantly by project maintainers (**104** of **158**), with the remainder from contributors (**26**), other commenters (**18**), and the original poster self-answering (**10**). Threads span **2020–2026** and contain a median of **4** comments (mean **5.7**, max **41**).

Of the **158** verified pairs, **105** (**66.5%**) satisfy the additional *hard-verifiability* requirement: the answer states at least one externally checkable fact. This hard subset is the part of the benchmark that supports fact-grounded automated grading (the remaining **53** pairs are retained for taxonomy construction so that the taxonomy is not biased toward only those question types that yield checkable identifiers). Across the hard subset, the most common fact types are fixed version numbers (**47** pairs) and advisory URLs (**44**), followed by fix PRs (**26**), CVE identifiers (**24**), fix commits (**21**), and GHSA identifiers (**14**). Hard facts are spot-checked against NVD, GHSA, and OSV at construction time, and the advisory-database snapshot date is recorded with the release to support future re-verification.

D. Artifact-Type and Hard-Fact Annotation

Each verified pair carries an `artifacts_needed` list recording which artifact types the maintainer's answer drew on, and a structured `hard_facts` record. The artifact-type distribution (Table II) shows that answering these questions is rarely a matter of reading source code alone: documentation and project source dominate, but advisories, pull-request and commit history, dependency manifests, and external references (RFCs, registries, upstream issues) each appear in a non-trivial fraction of answers. This spread is what motivates the per-artifact context conditions in the evaluation (§III): different question categories lean on different artifact types, and the annotation makes that dependency measurable.

E. Open Coding Toward a Question Taxonomy

To characterize *what* developers ask about, we apply inductive open coding to the verified pairs. The coding unit is the verbatim Q&A exchange. An LLM coder (`gpt-5.4-mini`, accessed through a uniform LiteLLM interface) is prompted to assign one or two short noun-phrase codes naming the security concern at issue, given the issue title and body, the Q&A summary, the full thread, and the focal question and answer. As with detection, an LLM-assisted first pass makes coding tractable and consistent at corpus scale while keeping a human in control of the final code: the first author then reviewed every

TABLE I

REPOSITORIES IN THE CURRENT SECDEVQA CORPUS. **PAIRS** = Q&A PAIRS RETAINED AFTER MANUAL VERIFICATION; **HARD** = SUBSET WHOSE ANSWER CARRIES AT LEAST ONE EXTERNALLY VERIFIABLE FACT (CVE/GHSA ID, FIXED VERSION, FIX COMMIT/PR, ADVISORY URL). STARS ROUNDED TO THOUSANDS (**2026-06-08**).

Repository	Lang.	Stars	Pairs	Hard
psf/requests	Python	54.0k	24	14
axios/axios	JS	109k	23	19
expressjs/express	JS	69.1k	23	17
pyca/cryptography	Python	7.6k	21	13
urllib3/urllib3	Python	4.0k	14	6
stripe/stripe-node	JS	4.4k	13	5
python-pillow/Pillow	Python	13.6k	12	12
rails/rails	Ruby	58.5k	11	9
auth0/node-jsonwebtoken	JS	18.2k	9	5
fastapi/fastapi	Python	99.0k	8	5
Total (10 repos)			158	105

pair in the same annotation interface, free to edit or replace any LLM-assigned code, so the resulting codebook reflects human judgment rather than model output. The first-author verification note is deliberately withheld from the coder to avoid anchoring its codes to the acceptance rationale.

This pass produced **261** code instances over the **158** pairs (**1–2** per pair), with **235** distinct codes—a fine-grained vocabulary that reflects how specific developer security questions are (e.g. "missing subject alternative names," "HTTPS redirect scheme downgrade," "transitive dependency vulnerability"). The single most frequent code, *transitive dependency vulnerability*, accounts for **12** instances; the long tail of singleton codes confirms that the corpus is not dominated by a handful of canned questions.

The next step is axial coding: collapsing the open codes into a smaller set of higher-level categories that will anchor the per-category analysis in RQ2–RQ3. Table III presents a *preliminary, author-proposed* grouping of the **235** codes into **eleven** candidate categories, reported here to convey the shape of the emerging taxonomy. The two largest groups—HTTP request and response security (SSRF, redirects, headers, cookies) and TLS and certificate validation—together cover roughly half the corpus, consistent with the HTTP-client and web-framework projects mined so far. We stress that this grouping is not final: category names and boundaries will be fixed through axial coding with an independent second coder, and we will report inter-rater agreement (Cohen's κ) on a held-out sample, together with a saturation analysis as the corpus grows.

III. EXPERIMENTAL SETUP

A. Research Questions (RQs)

RQ1: What categories of security concerns motivate developer questions in open-source projects, and which artifact types do maintainers draw on to answer them?

TABLE II

ARTIFACT TYPES THE MAINTAINER'S ANSWER DREW ON, ACROSS THE **158** VERIFIED PAIRS (A PAIR MAY DRAW ON SEVERAL). ANNOTATIONS ARE LLM-EXTRACTED AND SPOT-CHECKED; THEY DRIVE THE CONTEXT CONDITIONS OF THE EVALUATION.

Artifact type	Pairs	%
documentation (docs, README, changelog)	126	79.7
code (repository source)	92	58.2
advisory (GHSA / OSV / project advisory)	35	22.2
pr_data (PR diff or discussion)	23	14.6
issue_tracker (linked issues)	23	14.6
dependency_manifest	19	12.0
external_reference (RFC, registry, etc.)	14	8.9
commit_history (log, blame, commit)	14	8.9
cve_cwe_db (NVD/CWE/CVSS)	7	4.4
ci_logs	2	1.3

RQ2: How accurately do frontier LLMs, open-weight LLMs, and coding agents answer real developer security questions, broken down by question category?

RQ3: Which artifact types are necessary to answer which categories of developer security questions, and what is the marginal accuracy contribution of each artifact type when provided as context?

RQ4: When agents have autonomous tool access to a repository, do they retrieve the artifacts actually needed to answer a given security question, or do they fail at artifact selection?

IV. THREATS TO VALIDITY

External Validity. We mine only publicly visible artifacts. Security issues are often disclosed privately and resolved under embargo, leaving just a released advisory or changelog entry in public; SecDevQA captures only questions negotiated in the open and may under-represent privately-routed ones. We scope our claims to publicly resolved security Q&A. Repositories are chosen for security activity and popularity (**three** languages so far), and the hard-verifiable filter may exclude question types lacking checkable facts. We report per-repository results and derive the taxonomy from the full corpus to keep both gaps visible.

Internal Validity. LLM-assisted pipeline. [TO BE DONE] Free-text grading uses an LLM judge. Hard facts ground but do not eliminate judge error; we verify named claims, cross-check identifiers, and re-evaluate a **15%** sample by hand. Public threads may be memorized [5]; the no-context condition with training-cutoff stratification flags this.

V. RELATED WORK

REFERENCES

[1] Y. Acar, M. Backes, S. Fahl, D. Kim, M. L. Mazurek, and C. Stransky, "You get where you're looking for: The impact of information sources on code security," in *2016 IEEE Symposium on Security and Privacy (SP)*, 2016, pp. 289–305.

TABLE III

Preliminary, author-proposed AXIAL GROUPING OF THE OPEN CODES INTO CANDIDATE HIGHER-LEVEL CATEGORIES. **PAIRS** COUNTS VERIFIED PAIRS TOUCHING THE CATEGORY (A TWO-CODE PAIR MAY FALL IN TWO CATEGORIES, SO THE COLUMN SUMS TO MORE THAN **158**). THIS GROUPING IS A WORKING DRAFT TO BE FINALIZED BY AXIAL CODING WITH AN INDEPENDENT SECOND CODER AND INTER-RATER AGREEMENT; CATEGORY NAMES AND BOUNDARIES ARE NOT YET FIXED.

Candidate category	**Pairs**	**Representative open codes**
HTTP Request and Response Security	**40**	raw request body, credential leakage on redirect, CSRF token generation, double submit cookie, automatic redirect handling, absolute URL bypass
TLS and Certificate Validation	**38**	SSL certificate verification failure, OpenSSL version mismatch, ASN.1 parsing error, TLS SNI hostname handling, certificate verification failure, stricter SSL verification
Dependency and Supply-Chain Security	**26**	transitive dependency vulnerability, transitive dependency CVE, compromised package versions, dependency version update, prepare script install failure, false positive dependency vulnerability
Authentication and Session Integrity	**17**	webhook signature verification, unsigned JWT handling, question regarding jwt signing, signature verification bypass, token blacklist, JWT logout invalidation
Cryptographic Key and Algorithm Use	**15**	SM2 key algorithm mismatch, RSA key length requirement, cryptographic algorithm default mismatch, post-quantum support roadmap, encrypted RSA key loading, silent hash digest change
Input Handling and Denial of Service	**15**	memory exhaustion DoS, unsafe boundary generation, ReDoS vulnerability, ReDoS protection, regex route matching, array index limit
Vulnerability Report Triage	**14**	false positive report, scanner false positive, false positive vulnerability report, false vulnerability report, vulnerability database error, advisory database update
Patch, Release, and Compliance	**14**	fixed version announcement, SSRF vulnerability backport, safe version guidance, patched version marking, TLS deprecation support, SSRF fix availability
Secure Configuration and Defaults	**13**	deprecated URL parsing, opt-in security setting, missing config types, android network security config, initializer order misconfiguration, missing rate limiting
Platform and Version Compatibility	**10**	RAT on machine, issue tracker reference, local version parsing, Windows Python compatibility, permission denied error, missing attribute regression
Concurrency and Data Integrity	**7**	global state mutation, thread safety data race, shared block cache, deadlocked transaction state, writes outside transaction, thread safety issue

TABLE IV

COMPARISON WITH RELATED WORK. **SEC.** = SECURITY-SPECIFIC CORPUS; **ADV.** = EXTERNAL ADVISORY ARTIFACTS AS CONTEXT; **GRD.** = FACT-GROUNDED GRADING; **ABL.** = ARTIFACT-TYPE ABLATION. ✓ FULL; P PARTIAL; × NONE.

Paper	**Sec.**	**Adv.**	**Grd.**	**Abl.**
SWE-QA [3]	×	×	P	×
StackRepoQA [5]	×	×	×	×
CoReQA [4]	×	×	P	×
SecVulEval / CodeSecEval	✓	×	✓	×
SecureAgentBench	✓	×	✓	×
SecQA	✓	×	✓	×
SecDevQA (ours)	✓	✓	✓	✓

[2] F. Fischer, K. Böttinger, H. Xiao, C. Stransky, Y. Acar, M. Backes, and S. Fahl, "Stack overflow considered harmful? the impact of copy&paste on android application security," in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 121–136.

[3] "Swe-qa: Can language models answer repository-level code questions?" Preprint, arXiv:2509.14635, 2025.

[4] "Coreqa: Uncovering potentials of language models in code repository question answering," Preprint, arXiv:2501.03447, 2025.

[5] "Beyond code snippets: Benchmarking llms on repository-level question answering," Preprint, arXiv:2603.26567, 2025.

[6] "Secvuleval: Benchmarking llms for real-world c/c++ vulnerability detection," Preprint, arXiv:2505.19828, 2025.

[7] "Is your ai-generated code really safe? evaluating large language models on secure code generation with codeseceval," Preprint, arXiv:2407.02395, 2024.

[8] "Secureagentbench: Benchmarking secure code generation under realistic vulnerability scenarios," Preprint, arXiv:2509.22097, 2025.

[9] Z. Liu, "Secqa: A concise question-answering dataset for evaluating large language models in computer security," Preprint, arXiv:2312.15838, 2023.